CS3213 Foundations of S. Engineering (FSE)

AY24/25 Sem 2. github.com/gerteck

- Software development processes (agile vs. plan-driven)
- Requirements engineering, Modeling, Software architectures
- Software testing, Static analysis, Debugging, fault localization.

1. Software Engineering

Concerned with all aspects of software production, initial conception to operation and maintenance. Team development of multiversion programs. People at core, code important but only small part of building product.

- **Software engineering**: *practicalities* of developing, delivering useful software. Programming integrated over time. Software sustainability to keep software evolvable over time. People as first-order success driver.
- **Origins**: 1965s software crisis, Margaret Hamilton coins software eng. Bugs cost - 2022 US $2.41 trillion, technical debt biggest obstacle to existing codebases. E.g. crowdstrike 2024 outage, DBS 2023, Java Log4J 2021 zero-day vulnerability. Fred B. – no silver bullet (M. Man-Month).
- **Complexity**: *Essential complexity* - inherent, e.g. work w. stakeholders to determine product. *Accidental complexity* - solvable, e.g. buffer overflow exploitation protection. Hardest single part – deciding what to build.
- **Techniques to address key challenges**: Address users' needs with software, correctness and reliable, efficiency with limited resources.

Central Software Engineering Activities

Software Specification: Define functionality, constraints on operation.
Software Development: Implement to meet specification.
Software Validation: Ensure performance & requirements met.
Software Evolution: Adapts the software to evolving customer needs.

"Shifting Left" - more costly to address a defect late in the process. SE processes have been "shifting left".

SWE Qualities

- **Personal Pinciples**: Humility (Recognize limitations, self-improvement), Respect, Trust. Thrive in ambiguity, value feedback, challenge status quo, users first. Improving, Passionate, Knowledge about People/Org, Sees Forest & Trees, Create shared context, Creates safe haven. Working alone increases risks, design mistakes, irrelevance, and "bus factor."
- **Software Product Attributes**: *Elegant*: Simple, intuitive designs: easy to understand, maintain. *Anticipates Needs and future requirements*.

2. Software Engineering Processes

Software process model: rep. of software process (sequence of activities). Illustrates different approaches to developing software.

- **Waterfall Model**: Simple, linear plan-driven software process model. Output of one phase feeds into the next. Development starts only after req. eng and design. **Pros/Use Cases**: Embedded-hardware /Safety-critical /Large systems. Works well if requirements clear, large projects, or distributed teams. **Cons**: accommodate change difficult.
- **Incremental Development**: Overlapping phases of specification, development, and validation. Built in increments, adding functionality. Can be plan-driven, agile, or mix. **Pros**: Lower cost of accommodating change, faster delivery. **Cons**: Harder to measure progress, may lead to system degradation (require refactoring).
- **Integration and Configuration / Reuse-based**: Reuse existing components/framework/library over from scratch. May need redefine reqs. based on identified components. **Pros**: Reduce amt. to develop, lower cost, risk. **Cons**: Limit customizatn, third-party dependency, integration complexity.

Agile Software Engineering

Agile frameworks (e.g. scrum) not specific to software dev. Designed for changing, unclear requirements and incremental dev, deliver value quickly.

- **Agile**: Both mindset and concrete mngmt engineering framework. Focus on deliver value, avoid waste. Feedback loops, customer involvement. Adaptive to change. Lightweight process, empower people, lean manufacturing, buzzword certifications.
- **Historical Development and Origins**: 1980-90s: Trad. planning/dev processes. Late 1990s: Dissatisfaction, agile as alternative to rigid waterfall. 2001: Agile Manifesto by reps from frameworks like Scrum, XP, Feature-Driven Development (FDD). Influenced by *lean manufacturing*, *The New New Product Development Game (1986)* – overlapping stages, built in instability, subtle control, self-organizing teams (autonomy, self-transcendence of teams, cross-fertilization, learning & learning transfer cross teams).
- **Lean software development**: Limit waste (hardship in daily work) – partially done work, non-value adding processes/features, task-switching, waiting for input, defects, handoff motion.
- **Manifesto Principles**: Individuals & interactions over processes. Working software over compreh. docs. deliver frequently. Customer collab over contract nego (comms daily), Respond to change over following plan (wlc. changing req.). Working software as progress measure, sustainable development. technical excellence and good design enhances agility, simplicity is essential. regular reflection.
- **Agile Adoption** : Larger the org, more likely to use hybrid mode. Bigger teams likely still use waterfall. Agile has concrete benefits - better collaboration, alignment to business needs, better quality software. **Problems**: business side slow to embrace. Too many mixed systems, siloed teams delays. Incorrect practice, managment buy-in, pain of lacking documentn.

Scrum

'Agile synonym'. General, lightweight framework for project management.

- **Core Principles**: Empirical process control and lean thinking. Focuses on delivering value through increments in sprints.
- **Scrum Pillars**: Transparency (work and process visibility), Inspection (frequent evaluation), and Adaptation (adjustments as needed).
- **Scrum Team**: Self-managing, no hierarchies, typically $\leq$ 10members. *Scrum Master*: Facilitates Scrum process, removes impediments. *Product Owner*: Maximizes product value, manages Product Backlog. *Developers*: Responsible for creating Increment each Sprint.
- **Scrum Events**: *Sprint*: Timeboxed iteration ($\leq$ 1 month), deliver working Increment. *Sprint Planning*: Defines Sprint Goal, selects backlog items. *Daily Scrum*: 15-minute daily sync to track progress and adjust plan. *Sprint Review*: Team present results, adjust Product Backlog. *Sprint Retrospective*: Reflect on improvements for future Sprints.
- **Scrum Artifacts**: *Product Backlog*: Ordered list of work items aligned with Product Goal. *Sprint Backlog*: Selected items for Sprint w. actionable plan. *Increment*: Deliverable output meeting defn. of Done.
- **Timeboxing**: Sprint(S): $\leq$ 1 mnth. S. Planning: $\leq$ 8 hrs. Daily Scrum: 15 minutes. S. Review: $\leq$ 4 hrs. S. Retrospective: $\leq$ 3 hrs.
- **Kanban Board**: Often used in Scrum to visualize workflow. It helps teams track progress, limit work in progress (WIP), and improve flow (movement of potential value through a system).
- **Definition of Workflow (DoW)**: A shared understanding of how work items flow through the system. Includes *Units of value* (work items or user stories), defined states (stages) work items flow through, policies for how items move through each state. WIP limit controls enforced.

Extreme Programming (XP)

Highly influential agile methodology (late 1990s). "extreme" approach to iterative development and best practices. New ver may built several times/day, increments every 2 wks, All tests successful for accepted builds.

- **Pair Programming**: One dev programming, other review, focus on big picture. (driver & navigator). Both can think and interact at same level of interaction. If one greater expertise likely to dominate interaction. Switch roles after a time period, and switch partners frequently. *Pros vs Cons*: Fewer bugs, better code understanding, peer learning, but lower cost / scheduling efficiency, personal clash.
- **Collective Ownership**: Everyone responsible for all code, anyone can change any code. Pair prog facilitates this, as everyone better understands larger system, contribute to different parts of code.
- **Test-driven development**: Unit tests written before code, test all potential fail cases. helps development, facilitates other CI, refactoring etc.
- **Continuous Integration**: Dev team work on latest ver. Integrate code to main branch freq. Facilitate using unit tests. (GH Actions, Jenkins).
- **Simple Design, Refactoring**: "simple is best", refactoring as part of the lifecycle, keep it maintainable, easier to extend.
- Others: on-site customer for access system requirements, sustainable pace for net effect on code quality / long term productivity.

DevOps

In incremental development, challenge software deployment quickly & reliably. Involves collaboration btw. dev (scrum teams) & operations teams.

- **DORA (DevOps Research and Assessment)**: Framework for measuring and improving DevOps performance through key metrics like deployment frequency, lead time for changes, change failure rates, mean time to recovery. *Observability and Monitoring* for measuring DevOps performance. Monitoring of running code and infrastructure provides insights to optimize the value stream.
- **Silos**: Traditionally, dev and ops teams separated, conflicting goals. Developers focus on delivering features quickly, while operations prioritize stability and uptime. DevOps bridges gap by fostering collaboration.
- **DevOps Overview**: Unified approach, emphasize automation, collaboration, continuous delivery. Popularized 2010, aims for routine, predictable deployments. *DevOps vs. Agile*: Agile focus on iterative dev, deliver value, DevOps extends by automating deployment pipeline. Emphasis on CI, delivery, feedback loops. "Infinite loop" of dev and ops.
- **DevOps Principles**: *Flow*: Reduce batch sizes & handoffs (work passing across depts/teams) to streamline delivery. *Feedback*: Fast feedback loops to detect, resolve issues early. *Continuous Learning*: Promote experimentation, learning, mechanisms to share knowledge. Safety culture.
- **Learning from Incidents**: Encourages *blameless postmortems* to investigate failures, implement preventive measures. Fosters culture of experimentation and learning.
- **Value Stream Mapping**: Technique to visualize and analyze the flow of work, identifying inefficiencies and improving delivery processes. (E.g. develop unit test $\rightarrow$ code solution $\rightarrow$ PR reviews $\rightarrow$ QA $\rightarrow$ regression, performance test $\rightarrow$ deploy $\rightarrow$ smoke test $\rightarrow$ UAT test)
- **Continuous Integration and Deployment (CI/CD)**: Automates the integration, testing, and deployment of code changes, enabling frequent and reliable releases (push to production). *Infrastructure as Code*: Treats infrastructure configuration like code, using version control, automation to manage servers and environments.

3. Software Requirements Engineering

Functional vs. Non func. Req. Quality Attribute Scenarios. Elicitation & Analysis. Documentation. Validation. Process Models. Change Mngmt.

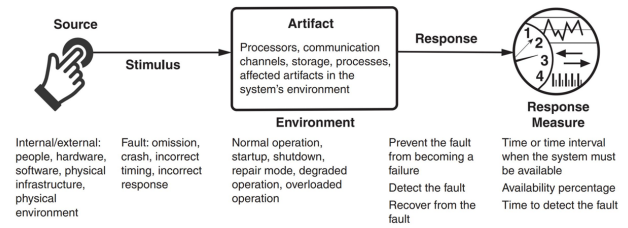
- **Requirements** describe services system should provide, constraints on its operation. Reflect customer needs for systems w certain purpose e.g. device control, order placement, or information retrieval.
- **Requirements Engineering (RE)**: Process of discovering, analyzing, documenting, and validating these services and constraints. RE focus on problem space, not solution space.
- **Wicked Problem**: No definitive formulation or stopping rule, solutions not true/false but better/worse, no immediate/ultimate test for solutions. Every solution is a "one-shot operation" with significant consequences. Each problem is unique, often a symptom of another problem. No exhaustive set of potential solutions or permissible operations. Social planners are accountable for the consequences of their actions.

Requirement Engineering (RE), Why & Challenges

- **High Cost of Errors**: Errors during requirements phase costly.
- **Importance of RE**: Facilitates communication between stakeholders. (*Customer Relations and Communication*). Provides the foundation for defining structure, behavior of software system. Serves as basis for testing, accepting final product. (*QA, Acceptance*) Dictates quality of system, risks, and operational overhead. (*Maintenance and Evolution*)
- **RE Challenges**: Difficult to determine if all relevant requirements are collected. Requirements engineering is resource-intensive, often one-shot effort. Requirements cannot be generalized.
  - *User-Related*: conflicted views, incomplete understandg of needs, limits.
  - *Analyst-Related*: Poor problem domain knowledge, omit "obvious" info.
  - *Communication*: Requirements evolve w. time, Ill-defined system boundary. Vague, untestable requirements ("user-friendly," "robust").
- *NaPiRE* (RE pains): Incomplete/hidden req, comm. flaws, time-boxing (not enough time), moving targets, inconsistent/underspecified req...

Functional vs. Non-Functional Requirements

- **Functional Requirements**: Describe what system should do. Stakeholders typically focus on functionality, not determine system architecture. (any functionality implementable using various designs.)
- **Non-functional Requirements**: Define system qualities, not specific services (e.g., performance, security, usability). Often more critical than FRs, may involve implicit stakeholder expectations. Can conflict (e.g., security vs. usability). Influence architectural tradeoffs (e.g., *quality attributes*).
- Non-functional requirements should be quantitative to be testable. E.g. ("throughput suff. high." vs. "throughput to exceed 1,000 query/sec")
- **Quality Attribute Scenarios**: framework to clarify, document NFRs. Up to 6 elements: (*Stimulus*: An event arriving at the system. *Source*: Origin of stimulus (e.g., human, system), *Artifact*: part of system affected by stimulus, *Response*: system's reaction. *Response Measure*: Quantifiable metrics for response. *Environment*: scenario context).



Other Non-Functional Requirements and Metrics

- **Usability**: How easily users accomplish tasks, support provided. *Metrics*: Task time, errors no., learning time, satisfaction.
- **Availability**: System readiness to perform tasks, including reliability and recoverability. *Metrics*: Availability %, time to detect/repair faults, etc.
- **Performance**: System responsiveness and throughput. *Metrics*: Latency, throughput, jitter, resource usage, etc.
- **Modifiability**: The cost and effort of making changes to the system. *Metrics*: Effort, elapsed time, new defects introduced, etc.
- **Deployability**: The ease of deploying and rolling back system changes. *Metrics*: Cost, percentage of failed deployments, cycle time, etc.
- **Energy Efficiency**: System efficiency in energy usage (e.g IoT, AI).
- **Security**: Protecting data and systems from unauthorized access. (CIA Triad) *Metrics*: Accuracy of attack detection, size of trusted execution
- **Data Security and Privacy**: PDPA (Singapore), GDPR (EU).
- **Safety**: Avoiding system states that cause harm or damage. *Metrics*: Percentage of unsafe states avoided, recovery time, etc.

Stakeholders and Requirements Engineering (RE) Activities

RE processes vary based on the organization and project type.
3 Cs of user stories map to key activities:
Elicitation and analysis → Conversation (w. client).
Specification → Card (description).
Validation → Confirmation (acceptance tests to determine completeness).

Agile vs. plan-driven approaches overlap in elicitation but differ in emphasis: Agile focus on iterative feedback, collaboration. Plan-driven emphasize detailed specification and validation.

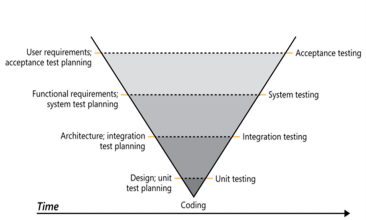
SRS: Software Requirements Specification: Central document, official statement of what should be implemented. May include user requirements, detailed specification of system requirements. Output of RE phase.

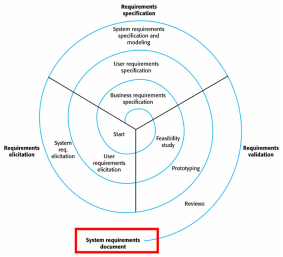
Stakeholders: person/org impacted / influencing system's req. *Identify stakeholders* as first step in RE, determines whose needs /desires addressed. Start with "baseline" stakeholders (e.g., users, decision-makers). Explore network to identify add. stakeholders.

- **Elicitation and Analysis**: Work w. stakeholders to get requirements. Elicitation techniques: (Closed/Open Interviews, Existing Docs Analysis – prior research reduces interaction time, (user-story) workshops – multiple stakeholders & resource intensive, FGs (users), prototyping, observe – discover implicit req. reflecting actual ways, personas). Supports user stories.
- **Specification**: Convert req. into standardized form (user-centric view). *User Stories*: represent. customer requirements (<As 'usertype', want 'goal', to 'reason'.>) Three Cs - card, convo, confirm. Epics – too large user stories to be split later. Include notes, NFRs (constraints). *Use Cases*: interaction btwn actor & system (e.g. check in flight). UML → use case diagram. May encompass multiple scenarios (single instance of usage e.g normal flow, crash, others etc). Natural Language – use standard format, consistency, associate rationales, avoid jargon.
- **Validation**: Ensuring requirements accurately define the desired system. (Agile: freq comm. w stakeholder, Plan: unclarities in req. document). Techniques: *Informal Reviews*: Peer review, colleague reviews requirements. Informal feedback collection (e.g., deskchecks, passarounds, walkthroughs). *Formal Reviews/Inspections*: Systematic analysis by a team of reviewers, roles: Moderator, reader, inspectors. Techniques: Defect checklists, requirement-by-requirement reviews.
- **Requirement document checks**: *Validity*: reflect real user current needs. *Consistency*: Avoid contradictory req. *Completeness*. *Realism*: feasible, within budget, tech constraints. *Verifiability*: testable.

Other Requirement Engineering Validations

- **Feasibility Study**: short study to assess: alignment with organizational objectives, budget and schedule, Integration with existing systems.
- **Prototyping**: Develop quick version of the system to validate requirements and design decisions. Reduces post-delivery changes by allowing users to experiment early.
- **Use Cases vs User Stories** (Use Cases – Plan-Driven): Specify functional requirements and serve as test bases. (User Stories – Agile): Act as placeholders for implementation, elaborated into acceptance tests.
- **Requirements as a Basis for Testing**: Requirements should be testable and used to derive test cases. User stories, cases as foundations for acceptance tests.
- **V-Model and Testing**: In plan-driven approaches, requirements validation aligns with testing phases in the V-Model.





Requirement Engineering Process Models

Agile Approaches: Agile avoids detailed requirements documents due to rapid changes. Relies on user stories and frequent stakeholder feedback. Short supporting documents may still define business and dependability requirements.
Plan-Driven Approaches: Follow a structured process model. RE phase is completed before implementation. Produces a requirements document, often part of the development contract.

- **User vs. System Requirements**: UR: High-level, natural language descriptions of system services and constraints. SR: Detailed descriptions of functions, services, and operational constraints.
- **RE Spiral Model**: iterative, plan-driven RE process model. Expands requirements incrementally, focusing on high-level business needs early and detailed system requirements later. Outputs SRS. Can incorporate agile development in later iterations.
- **RE Elicitation and Analysis Model**:
  Requirements Discovery (Interact w. stakeholders to uncover needs) → *Classification and Organization* (Group req. to coherent clusters) → *Prioritization and Negotiation* (Resolve conflicts, prioritize req) → *Documentation* (Record requirements for next iteration) → (repeat)
- **Requirements Change Management**:
  Established Plan for to manage evolving requirements.
  Unique identification of requirements (to cross-reference).
  Established change management process to assess impact and cost.
  Traceability policies to link requirements to design.
  Tool support for managing requirements

4. System Modeling and Software Architecture

Abstractions. Modeling Perspectives. UML. C4. Different Architecture Structures, Drivers, Patterns, Tactics.

Systems Modeling

Develop abstract models of a system, usually some graphical notation based on diagram types in Unified Modeling Language (UML).
Means to an end (developing software).

- Emphasized in plan-driven development, also used in agile in lightweight manner ("just enough" design).
- Formal models (mathematical) can be developed for auto verification.
- Models are created during requirements eng, system design, and post-implementation documentation.
- **Abstraction vs Representation**: Abstraction simplifies systems, omits details. Representation maintains all system information.
- **Different perspectives**: External, Interaction, Structural, Behavioral.

Modeling Languages

Unified Modeling Language (UML, most known, rich tool support, stand. notn), C4, BPMN, SysML, etc.

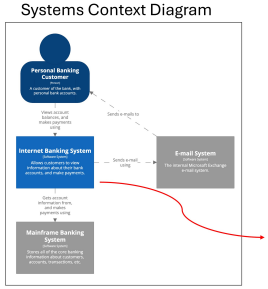
UML Key Diagrams:

- Class Diagrams: Show object classes and associations.
- Sequence Diagrams: Show interactions btw. actors & components.
- Activity Diagrams: Show process or data flow activities.
- State Diagrams: Show system reactions to events.
- Use Case Diagrams: Show interactions btwn system & environm.

C4 Model: Agile Architectural Modeling. For software architectures. Do "just enough" design / upfront thinking for starting point/ general direction.

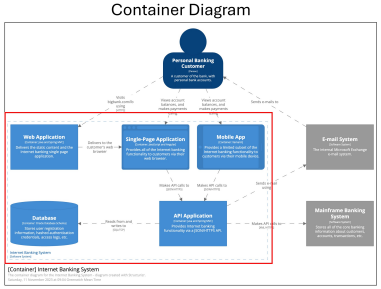
- Hierarchical architectural modeling (4Cs): System Context, Containers, Components, Code. Notation Independent.
- *System Context diagram* → starting point, how software fits into world.
- *Container diagram* → software system, high level building blocks.
- *Component diagram* → individual container, decomposes containers to identify major structural building blocks (i.e., components). Not recommended unless level of detail adds value.
- *Code diagram* (e.g. UML class) → indivdual component implementation. Not recommended for C4 / Agile context use.

Systems Context Diagram



Big picture of the system landscape, focusing on people and software systems

Container Diagram



A container diagram zooms into the system boundary. Container is separately runnable/deployable component e.g. web app, mobile app, database or file schema

Software Architecture

The overall structure of a system, including components, relationships, and properties. How devs explain system at high level.

- S. Architecture vs S. Design: Software Architecture is part of design, focusing on high-level structures. Design (e.g. detailed class design)
- Influences quality attributes, modifiability, and communication among stakeholders. Mitigate highest priority risks. Difficult to change later.
- **Architectural Drivers**: Architecturally Significant Requirements (ASRs): Requirements that significantly impact architecture. Non-functional req. (e.g., availability, performance) are often ASRs. Functionality must have major influence on architecture. (E.g. the system should provide five nines (99.999 percent) availability).

Often, system architec. modeled informally w. simple block diagrams.

Architectural Reasons and Consequences

- **Quality Attributes**: Architecture significantly influences whether system meets quality attributes (non-functional requirements). (E.g. *High Performance* – require managing time-based behavior and shared resource access, *Safety and Security...*)
- **Modifiability and Change**: Modifiability is a critical quality attribute. Requires assigning responsibilities to components to limit the impact of changes. *Categories of changes*: Local – affect only single component, Non-local – Affect multiple components (e.g. UI). Architectural Change – Affects entire system (e.g., single-thread to multi-thread). Good architecture ensures most changes local or non-local.
- **Communication Among Stakeholders**: Architecture as common abstraction for mutual understanding, negotiation, and consensus.
- **Additional**: Enables reasoning of cost, schedule, basis for reusable models in product lines, focuses on component assembly, channels developer creativity by restricting design alternatives, foundation for training new team members, enable early prediction of system qualities, defines constraints for subsequent implementation, influences org structure (Conway's Law), supports incremental dev.
- **Conway's Law**: Organizations design systems that mirror their communication structures – Arch. often forms basis for work-breakdown structures.

Architecture Patterns and Tatics

Design is complex activity, architecture is a tradeoff of attributes.
Architectural tactic: design decision that influences a quality attribute.

- **Attribute-Driven Design (ADD)**: Systematic approach to architecture design. Each design iteration, address an ASR.
  Steps: Identify ASRs → establish iteration goals (satisfy set of ASRs) → refine structure → select designs → instantiate patterns → record decisions → analyze designs.

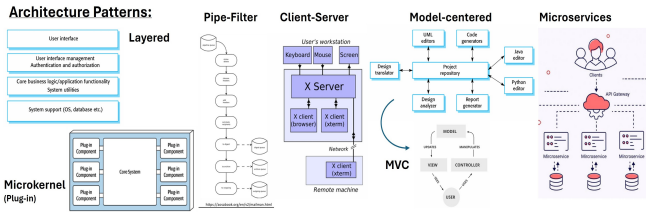
Architectural Patterns *Established architectural solution, may comprise multiple architectural tactics.* Often found useful over time/domains, documented and disseminated.

- **Antipattern – Big Ball of Mud**: Common in practice. Lack of architectural design, Business pressure, Erosion of architecture over time. Poor maintainability, extensibility (Quality Attributes)

Common Architectural Patterns

- **Layered Architecture**: Structures systems into layers with defined interfaces. Layer only uses layer directly beneath, through public interface. (constraints may be violated e.g. for performance, layer bridging, upper layer use deeper lower layer).
  Pros: portability (to diff env/OS), modifiability (lower layers changeable under hood, less interfaces to understand), reuse
  Cons: Layering possible performance penalty. If not properly designed, can impede (missing nec. low level abstraction needed at higher level).
- **Pipe-and-Filter**: Processes data through independent filters connected by pipes. *Filters*: processing components, take input, produce output, no assumption on other filters, stateless. *Pipes*: connect multiple filters with each other. (E.g. MailMan - add header, digest, archive, outgoing).
  Pros: modifiability (independent filters) and reconfigurability, evolution (add/combine filers differently)
  Cons: format data transfer must be set. performance as every filter must parse input convert to output format.
- **Model-Centered**: Independent components interact with a central model than each other. Repository style. (E.g. IDEs).
  Pros: components can be independent. highly modifiable. concurrency. State/data manged consistently (in one place).
  Cons: model/repo is single point of failure. change to model = many changes. Distributing repo can be difficult.

  MVC design pattern: "instantiation" of model-centered, goal of MVC to separate model (business logic) from UI (controller/view).
- **Microkernel (Plug-in)**: Base system with plug-in components (e.g., Eclipse SDK, web browsers).
  Pros: modifiability, extensibility, testability. Plugins can be developed, tested, evolved by diff. teams independently.
  Cons: can introduce security /privacy vulnerabilities
- **Client-Server**: central server, provide service to multiple distributed clients. Client request service, server cannot initiate. (E.g. web server)
  Discovery: client use discovery service, server respond w. agreed protocol.
  Interaction: client sends request, server process, respond.
  Pros: Low coupling btwn server clients, no coupling among clients. Easily scale no. clients and server. Both can evolve indptly.
- **Microservices**: Independently deployable services communicating via APIs. Typically stateless, small teams, small code bases. Service dependencies acyclic. Popularized by large companies. Goal - decoupling, microservices expected to be independent w own domain/workflow. typically smaller than services in SOA.
  Pros: quick time to market, indiv. deployability, test, scalability.
  Cons: network comm. overhead, complex transactions, challenge in design /maintain sea of microservices

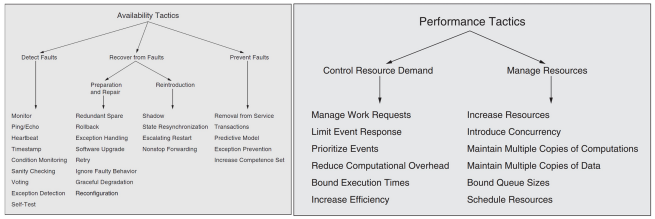


Monoliths vs. Service-Oriented Architecture (SOA): Monolith typically single deployable unit (diff. way of partition e.g. layered archi.). SOA for distributed-architecture styles, services separately deployed, only interact over interfaces, network. Related to microservices.

Architectural Tactics

Assign decision that influences a quality attribute. Tatics are design decisions that influence quality attributes. (modify existing patterns) E.g.

- *Availability*: Detect faults (monitoring, heartbeat), recover (redundant spare, rollback), prevent faults (ACID transactions, remove from service).
- *Performance*: Manage work requests, limit event response, maintain multiple copies of data, schedule resources.



"20 minutes rule": Continuous learning essential for staying updated.

5. Software Testing

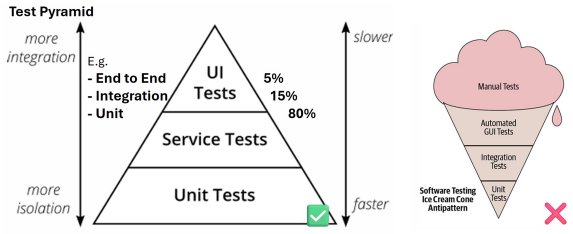
Test levels. Flaky tests. Specification-based/Structural/Mutation testing.

Purpose of Testing: *Defect Testing*: Find bugs in software. (Tests show presence, but never absence.) *Validation Testing*: Ensure system meets user requirements.

- **Verification vs. Validation**: *Verify*: Build product right, conforms to specification. *Validate*: Build right product, software meets user needs.
- **Test Case**: A test case consists of: *Test Input*: Arguments, system state, or actions, *Test Oracle*: Mechanism, determine test pass or fail.
- **Test Oracle**: Can check functional properties (e.g., correctness) or non-functional properties (e.g., performance, security).
- **Manual vs. Automated Testing**: *Automated*: Use tools to auto generate inputs and apply oracles (e.g., JUnit). *Manual*: Tester manually inputs test cases and acts as oracle. ('Traditional' - auto as in test automation).
- **Pesticide Paradox**: 'Every method used to prevent/find bugs leaves residue of subtler bugs which those methods ineffectual'. Diff testing methods have pros, cons. Use many, similar to pests build immunity. (Law of diminishing return).

Testing Levels

- **Unit Tests**: Test indiv. components (e.g., single method or class, e.g. testing a method converting AST to string). *Pros*: Fast execution, easy control & write (no additional setup of (e.g., no DB, web services).
  Cons: Lack realism, don't represent real system execution. May miss integration bugs, often require mocks to simulate ext. dependencies.
- **Integration Tests**: Test interactions btw. multiple compnt. Focus on integrating target cmpnt. w. external cmpnt. (e.g., ext DB, web service). (e.g. SQLancer test query plan converted to expected internal repnstn.)
- **System Tests**: Test entire system as whole. *Pro*: realistic test env. *Cons*: slower execution, harder to write (complex setup), prone to flakiness (non-deterministic results). (e.g: Ensuring SQLancer compatibility with database system.)
- **Test Flakiness**: Pass/fail non-deterministically. Low result confidence.
  Causes: Concurrency issue (e.g., race conditions), async waits (e.g., not properly waiting for server responses), test order dependencies (e.g., improper cleanup), test timeouts, resource leaks. (e.g, test fail from `ConcurrentModificationException` in multi-threaded code.)
- **Test Pyramid**: strategy for balancing test granularity. Many small, fast unit tests. Fewer integration, system tests. *Antipatterns*: Too many high-level tests, slow execution, maintenance challenges.



- **Test Sizes at Google**: Tests classified by size to for efficiency, determinism.
  Small: Single process/thread, no I/O, fast, deterministic.
  Medium: Multiple processes, allow localhost network calls.
  Large: No restrictions, slower, less deterministic.

Testing and Processes

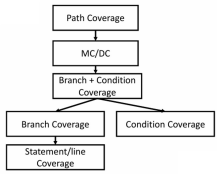
- **V-Model**: extension of waterfall model. Testing is planned alongside RE, archi, design phases. (E.g. acceptance test derived during RE).
- **User Accept Testing**: validate system meets user req, w customer participation. In *plan driven*, explicit phase after system implementation. In *agile*, UAT is interleaved (e.g., acceptance criteria in user stories).
- **Test-Driven Development (TDD)**: Emphasized by XP. Use as design process, focus 'happy path'. *Pros*: Focus on req, provide quick feedback. Encourages testable code, allows incremental exploration.
  Steps: Write test $\rightarrow$ ensure fails $\rightarrow$ simplest code to pass $\rightarrow$ refactor.
  When to Use: Useful for complex prob, unclear soln (incremental experimentation). Less useful if solution already clear.
  Schools of thought: (Classicist/Detroit, Inside-out start e.g. business rules $\rightarrow$ UI). (London/Mockist Outside-In: Start outside, use mocks).
- **Black-box vs. White-box Testing**: *Black-box 'specification testing'*: No internal info, based on requirements or function documentation. *White-box 'structural testing'*: Internal info, focus code structure, logic.

Black-box Testing

- **Specification-based Testing**: Derive from requirements (e.g., user stories, use cases, docs). Functional & non-functional requirements.
  Testing Workflow: Understand requirements (inputs, outputs), explore program (functionality), identify partitions (eqv classes of input/output), analyze boundaries, devise and automate test cases, augment test suite with creativity, experience on common bugs.
- **Equivalence Partitioning**: Divides inputs into equivalent classes (e.g., null, empty, single-char string). Output partn may help spot input partn.
- **Boundary Value Analysis**: Tests edge cases (e.g., min/max).

White-box Testing

- **Structural Testing**: use source code to guide testing. Catch requirement independent implementation aspect (e.g. cache, DSA). Focus coverage criteria. (e.g. Jacoco basic Java line coverage.)
- **Coverage Criteria**: *Line Coverage* (percentage executed). *Branch, Condition Coverage*: Percentage of branches (if [true/false], fors, whiles), $(a||b)$ executed. Condition coverage does not imply branch coverage. *Path Coverage*: Strongest criterion. Often impossible (e.g. loop).
- **Modified Condition/Decision Coverage (MC/DC)**:
  Test impt combinaton of cond, less cost.= $\frac{\#\text{conditions independently affecting decisions}}{\text{total \#conditions}}$.
  Every condition in a decision (e.g., if $(A\&\&B)$) takes every possible outcome (T and F). Ensures conditions independently affect decision outcome. Final set of tests, less than needed for path coverage. ($n+1$ test cases common lower bound for n conditions).



MC/DC: Modified Condition/Decision Coverage

Test case	A	AND	B	result	applying MCDC rules
1	1		1	1	A and B are decisive for the result "1"
2	1		0	0	B is decisive for the result "0"
3	0		1	0	A is decisive for the result "0"
4	0		0	0	This test case can be skipped based on the MCDC-rules, as both A and B have been decisive for both a true and a false result of the complex condition

- **Mutation Testing (White-box)**: Evaluate test suite quality by introducing bugs (mutants) and checking if tests detect them. (test case should 'kill' mutant - fail the test case). % of bugs detected = quality of test suite.
  Steps: Select statement in code, mutate code, run test suite, record outcome. Undo change, repeat. Calculate mutation score: $\frac{\#\text{killed mutants}}{\#\text{mutants}} \times 100$.
  (e.g. Pitest: Mutation testing tool for Java.).
  Pros: Identify undertested code.
  Cons: Comptn expensive; equivalent mutants may not change behavior.